
OpenVela 工程实践 • 上机实验

任务与线程上机实验

手册章节 3.2

DshanPI openvela Devkit • T113S3

姓 名张恒基

学 号2024210926

班 级2024211301

指导教师修佳鹏

提交日期2026 年 06 月 09 日

参考：《openvela 快速入门与工程实践（基于 T113S3）》

摘要

本报告记录 openvela 多任务系统第 3.2 节「任务与线程上机实验」的操作过程与运行结果。实验涵盖 hello 程序配置运行、pthread 多线程共享变量、FIFO 调度机制体验与 RR 时间片轮转调度配置，理解 Task/Thread 创建、调度优先级与双核 CPU 并发行为。

关键词：openvela；pthread；FIFO；RR 调度；任务与线程

目录

一、实验目的	1
二、实验环境	1
三、实验内容与操作过程	1
3.1 实验一：hello 程序配置与运行	1
3.2 实验二：创建第一个线程 (add + print)	2
3.3 实验三：FIFO 调度机制体验	3
3.4 实验四：RR 时间片轮转调度	4
3.5 实验五：查看任务信息	5
四、运行结果与分析	5
4.1 实验结果汇总	5
4.2 分析与思考	6
五、实验总结	6

一、实验目的

1. 掌握 openvela 中 hello 示例程序的 menuconfig 配置、编译烧写与 NSH 运行方法；
2. 使用 pthread_create 创建两个用户线程，观察共享全局变量 g_a 的累加与打印；
3. 体验 FIFO 调度：高优先级线程不休眠时低优先级线程与 shell 无法运行；
4. 配置 CONFIG_RR_INTERVAL=1 启用 RR 调度，对比时间片轮转后多线程交替运行现象；
5. 使用 ps、/proc/<pid> 查看任务状态、堆栈与调度策略。

二、实验环境

项目	配置
开发板	DshanPI openvela Devkit (T113S3, 双核 Cortex-A7)
操作系统	openvela / NuttX RTOS
开发环境	Ubuntu 虚拟机, 源码路径 ~/vela-opensource
串口工具	MobaXterm, 波特率 1500000, Flow Control: none
参考手册	《openvela 快速入门与工程实践》§ 3.2
主要源码	apps/examples/hello/hello_main.c

三、实验内容与操作过程

3.1 实验一：hello 程序配置与运行

操作步骤：

1. 进入工程目录并选择 lunch 配置：

```
cd ~/vela-opensource/vendor/allwinnertech/lichee/  
source vela_env.sh && source envsetup.sh  
lunch_nuttX    # 选择 2  
m
```

1. menuconfig 启用 hello 示例（搜索 /HELLO）：

```
cd ~/vela-opensource
./build.sh vendor/allwinnertech/boards/r528/r528s3-velaevb1/configs/
nsh/ menuconfig
```

1. 编译、打包、烧写镜像，串口执行：

```
cd ~/vela-opensource/vendor/allwinnertech/lichee/
m && pack
```

```
nsh> hello
Hello, World!!
```

【请插入截图】

#path

图 3-1 hello 程序运行串口输出

Listing 1: 图 3-1 hello 程序运行串口输出

1. (可选) 修改 rcS 自动启动：在 `init.d/rcS` 末尾添加 `hello &`，重启后观察自动运行。

3.2 实验二：创建第一个线程 (add + print)

将 `hello_main.c` 替换为手册 3.2.2 代码：创建 `add_thread`（每秒 `g_a++`）与 `print_thread`（每秒打印 `g_a`），主线程每 5 秒打印 `Hello, World!!`。

核心代码：

```
static volatile int g_a;

void *add_thread(void *arg) {
    volatile int *p = (volatile int *)arg;
```

```

    while (1) { (*p)++; sleep(1); }
    return NULL;
}

void *print_thread(void *arg) {
    volatile int *p = (volatile int *)arg;
    while (1) { printf("val = %d\n", *p); sleep(1); }
    return NULL;
}

int main(int argc, FAR char *argv[]) {
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, add_thread, &g_a);
    pthread_create(&tid2, NULL, print_thread, &g_a);
    while (1) { printf("Hello, World!!\n"); sleep(5); }
    return 0;
}

```

预期现象：串口交替输出 val = N 与 Hello, World!!, g_a 持续递增，说明两线程与主线程并发运行。

【请插入截图】

#path

图 3-2 双线程 + 主线程运行输出

Listing 2: 图 3-2 双线程 + 主线程运行输出

3.3 实验三：FIFO 调度机制体验

源码目录：source/3-2-3_FIFO 调度机制体验，替换 hello_main.c。

关键修改：

1. 三个线程（主线程、add_thread、print_thread）优先级均设为最高（225）；
2. 去掉 sleep(1)，改为空循环延时 for (volatile int i = 0; i < 100000000; i++);;
3. T113S3 仅 2 个 CPU 核，3 个不休眠的高优先级线程无法全部运行。

预期现象：

1. 主线程与 `add_thread` 可运行 (`g_a` 递增, Hello, World!! `g_a = N` 输出);
2. `print_thread` 无法运行 (无 `val =` 输出);
3. NSH 终端无法输入 (shell 被饿死)。

【请插入截图】

#path

图 3-3 FIFO 调度下 shell 无响应、print 线程饿死

Listing 3: 图 3-3 FIFO 调度下 shell 无响应、print 线程饿死

3.4 实验四: RR 时间片轮转调度

1. menuconfig 搜索 `/RR_INTERVAL`, 设为 1, 保存配置;
1. 使用 `source/3-2-4_RR 调度机制体验` 代码替换 `hello_main.c`;
1. 重新 `m && pack` 烧写运行。

预期现象: 启用 RR 后, 相同优先级线程按时间片轮流运行, `print_thread` 也能输出 `val = N`, NSH 终端恢复可用。

【请插入截图】

#path

图 3-4 RR 调度下三线程交替运行

Listing 4: 图 3-4 RR 调度下三线程交替运行

3.5 实验五：查看任务信息

运行 hello 程序后，另开串口或使用 & 后台运行，执行：

```
nsh> ps -heap 12
nsh> cat /proc/12/status
nsh> cat /proc/12/stack
nsh> cat /proc/12/heap
```

观察要点： 任务名、调度策略（SCHED_RR / SCHED_FIFO）、优先级、堆栈使用量。



Listing 5: 图 3-5 ps 与 /proc 任务信息

四、运行结果与分析

4.1 实验结果汇总

实验项	结果	现象说明
hello 配置运行	✓	NSH 执行 hello 输出 Hello, World!!
pthread 双线程	✓	g_a 递增，val 与 Hello 交替打印
FIFO 调度体验	✓	2 核仅 2 线程运行，print 线程与 shell 饿死

实验项	结果	现象说明
RR 调度体验	✓	时间片轮转后三线程均可运行
/proc 任务查看	✓	可查看优先级、调度策略、堆栈占用

4.2 分析与思考

1. Task vs Thread: openvela 中 Task 类似 Linux 进程（资源隔离），Pthread 是调度基本单元，同 Task 内线程共享全局变量、文件描述符与消息队列；
2. FIFO vs RR: `CONFIG_RR_INTERVAL=0` 时同优先级线程采用 FIFO，主动让出 CPU 前一直占用；设为 1 后 tick 中断触发时间片轮转，避免低优先级线程长期饿死；
3. 双核限制: T113S3 双核最多同时运行 2 个线程，第 3 个高优先级线程必须等待，这是理解嵌入式调度的重要实验现象。

五、实验总结

通过 3.2 节五项上机实验，掌握了 openvela 下 pthread 线程创建、优先级设置与 FIFO/RR 两种调度策略的实际表现。实验验证了多任务系统中「线程是调度基本单元、同 Task 内线程共享资源」的核心概念，为后续 MyWhackMole 多线程游戏开发与 SmartMole Pro 项目中的 LVGL / 音频 / 传感器多线程架构打下基础。

说明

截图文件请放入 `labs/images/` 目录，文件名与上文 `screenshot()` 路径一致，重新编译即可。